

第 2 章 循环结构程序设计

学习目标

- ☑ 掌握 for 循环的使用方法
- ☑ 掌握 while 和 do-while 循环的使用方法
- ☑ 学会使用计数器和累加器
- ☑ 学会用输出中间结果的方法调试
- ☑ 学会用计时函数测试程序效率
- ☑ 学会用重定向的方式读写文件
- ☑ 学会用 fopen 的方式读写文件
- ☑ 了解算法竞赛对文件读写方式和命名的严格性
- ☑ 记住变量在赋值之前的值是不确定的
- ☑ 学会使用条件编译指示构建本地运行环境
- ☑ 学会用编译选项-Wall 获得更多的警告信息

第 1 章的程序虽然完善，但并没有发挥出计算机的优势。顺序结构程序自上到下只执行一遍，而分支结构中甚至有些语句可能一遍都执行不了。换句话说，为了让计算机执行大量操作，必须编写大量的语句。能不能只编写少量语句，就让计算机做大量的工作呢？这就是本章的主题。基本思路很简单：一条语句执行多次就可以了。但如何让这样的程序真正发挥作用，可不是一件容易的事。

2.1 for 循环

考虑这样一个问题：打印 1, 2, 3, ..., 10，每个占一行。本着“解决问题第一”的思想，很容易写出程序：10 条 printf 语句就可以了。或者也可以写一条，每个数后面加一个“\n”换行符。但如果把 10 改成 100 呢？1000 呢？甚至这个重复次数是可变的：“输入正整数 n ，打印 1, 2, 3, ..., n ，每个占一行。”又怎么办呢？这时可以使用 for 循环。

程序 2-1 输出 1, 2, 3, ..., n 的值

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6     for(int i = 1; i <= n; i++)
7         printf("%d\n", i);
```

```

8   return 0;
9 }

```

暂时不用考虑细节,只要知道它是“让 i 依次等于 $1, 2, 3, \dots, n$, 每次都执行 `printf("%d\n", i);`”即可。这个“依次”非常重要: 程序运行结果一定是 $1, 2, 3, \dots, n$, 而不是别的顺序。

提示 2-1: `for` 循环的格式为: `for(初始化; 条件; 调整) 循环体;`

在刚才的例子中, 初始化语句是“`int i = 1`”。这是一条声明+赋值的语句, 含义是声明一个新的变量 i , 然后赋值为 1。循环条件是“ $i \leq n$ ”, 当循环条件满足时始终进行循环。调整方法是 `i++`, 其含义和 `i = i + 1` 相同——表示给 i 增加 1。循环体是语句“`printf("%d\n", i);`”, 这就是计算机反复执行的内容。注意循环变量的妙用: 尽管每次执行的语句相同, 但是由于 i 的值不断变化, 该语句的输出结果也是不断变化的。

提示 2-2: 尽管 `for` 循环反复执行相同的语句, 但这些语句每次的执行效果往往不同。

为了更深入地理解 `for` 循环, 下面给出了程序 2-1 的执行过程。

当前行: 5. `scanf` 请求键盘输入, 假设输入 4。此时变量 $n=4$, 继续。

当前行: 6. 这是第一次执行到该语句, 执行初始化语句 `int i=1`。条件 $i \leq n$ 满足, 继续。

当前行: 7. 由于 $i=1$, 在屏幕输出 1 并换行。循环体结束, 跳转回第 6 行。

当前行: 6. 先执行调整语句 `i++`, 此时 $i=2$, $n=4$, 条件 $i \leq n$ 满足, 继续。

当前行: 7. 由于 $i=2$, 在屏幕输出 2 并换行。循环体结束, 跳转回第 6 行。

当前行: 6. 先执行调整语句 `i++`, 此时 $i=3$, $n=4$, 条件 $i \leq n$ 满足, 继续。

当前行: 7. 由于 $i=3$, 在屏幕输出 3 并换行。循环体结束, 跳转回第 6 行。

当前行: 6. 先执行调整语句 `i++`, 此时 $i=4$, $n=4$, 条件 $i \leq n$ 满足, 继续。

当前行: 7. 由于 $i=4$, 在屏幕输出 4 并换行。循环体结束, 跳转回第 6 行。

当前行: 6. 先执行调整语句 `i++`, 此时 $i=5$, $n=4$, 条件 $i \leq n$ 不满足, 跳出循环体。

当前行: 8. 程序结束。

这个执行过程对于理解 `for` 循环非常重要: 语句是一条一条执行的。强烈建议教师在课堂上演示单步调试的方法, 并打开 i 和 n 的 `watch` 功能, 以帮助学生掌握如何用实验验证上面所介绍的执行过程。观察执行过程时应留意两个方面: “当前行”的跳转 (在 IDE 中往往高亮显示), 以及变量的变化。这二者也是编码、测试和调试的重点。根据实际情况, 教师可以用 IDE (如 Code::Blocks) 或者文本界面的 `gdb` 进行演示。`gdb` 的简明参考见附录 A。

提示 2-3: 编写程序时, 要特别留意“当前行”的跳转和变量的改变。

上面的代码里还有一个重要的细节: 变量 i 定义在循环语句中, 因此 i 在循环体内不可见, 例如, 在第 8 行之前再插入一条“`printf("%d\n", i);`”会报错^①。有经验的程序员总是尽量缩小变量定义的范围, 当写了足够多的程序之后, 这样做的优点会慢慢表现出来。

提示 2-4: 建议尽量缩短变量的定义范围。例如, 在 `for` 循环的初始化部分定义循环变量。

^① Visual C++ 6.0 等早期编译器允许在循环体之后访问 i , 但这样, 如果再写一个“`for(int i = 0; i < n; i++)`”则会出现 i 重定义的错误。

有了 for 循环，可以解决一些简单的问题。

例题 2-1 aabb

输出所有形如 aabb 的 4 位完全平方数（即前两位数字相等，后两位数字也相等）。

【分析】

分支和循环结合在一起时功能强大：下面枚举所有可能的 aabb，然后判断它们是否为完全平方数。注意，a 的范围是 1~9，但 b 可以是 0。主程序如下：

```
for(int a = 1; a <= 9; a++)
    for(int b = 0; b <= 9; b++)
        if(aabb 是完全平方数) printf("%d\n", aabb);
```

请注意，这里用到了循环的嵌套：for 循环的循环体自身又是一个循环。如果难以理解嵌套循环，可以用前面介绍的方法——在 IDE 或 gdb 中单步执行，观察“当前行”和循环变量 a 和 b 的变化过程。

上面的程序并不完整——“aabb 是完全平方数”是中文描述，而不是合法的 C 语言表达式，而 aabb 在 C 语言中也是另外一个变量，而不是把两个数字 a 和两个数字 b 拼在一起（C 语言中的变量名可以由多个字母组成）。但这个“程序”很容易理解，甚至能让读者的思路更加清晰。

这里把这样“不是真正程序”的“代码”称为伪代码（pseudocode）。虽然有一些正规的伪代码定义，但在实际应用中，并不需要太拘泥于伪代码的格式。主要目标是描述算法梗概，避开细节，启发思路。

提示 2-5：不拘一格地使用伪代码来思考和描述算法是一种值得推荐的做法。

写出伪代码之后，我们需要考虑如何把它变成真正的代码。上面的伪代码有两个“非法”的地方：完全平方数判定，以及 aabb 这个变量。后者相对比较容易：用另外一个变量 $n = a*1100 + b*11$ 存储即可。

提示 2-6：把伪代码改写成代码时，一般先选择较为容易的任务来完成。

接下来的问题就要困难一些了：如何判断 n 是否为完全平方数？第 1 章中用过“开平方”函数，可以先求出其平方根，然后看它是否为整数，即用一个 int 型变量 m 存储 $\text{sqrt}(n)$ 四舍五入后的整数，然后判断 m^2 是否等于 n。函数 floor(x) 返回不超过 x 的最大整数。完整程序如下：

程序 2-2 7744 问题（1）

```
#include<stdio.h>
#include<math.h>
int main()
{
    for(int a = 1; a <= 9; a++)
        for(int b = 0; b <= 9; b++)
        {
            int n = a*1100 + b*11; //这里才开始使用 n，因此在这里定义 n
```

```

int m = floor(sqrt(n) + 0.5);
if(m*m == n) printf("%d\n", n);
return 0;
}

```

读者可能会问：可不可以这样写？`if(sqrt(n) == floor(sqrt(n))) printf("%d\n", n)`，即直接判断 `sqrt(n)` 是否为整数。理论上当然没问题，但这样写不保险，因为浮点数的运算（和函数）有可能存在误差。

假设在经过大量计算后，由于误差的影响，整数 1 变成了 0.999999999，`floor` 的结果会是 0 而不是 1。为了减小误差的影响，一般改成四舍五入，即 `floor(x+0.5)`^①。如果难以理解，可以想象成在数轴上把一个单位区间往左移动 0.5 个单位的距离。`floor(x)` 等于 1 的区间为 [1,2)，而 `floor(x+0.5)` 等于 1 的区间为 [0.5, 1.5)。

提示 2-7：浮点运算可能存在误差。在进行浮点数比较时，应考虑到浮点误差。

另一个思路是枚举平方根 x ，从而避免开平方操作。

程序 2-3 7744 问题 (2)

```

#include<stdio.h>
int main()
{
    for(int x = 1; ; x++)
    {
        int n = x * x;
        if(n < 1000) continue;
        if(n > 9999) break;
        int hi = n / 100;
        int lo = n % 100;
        if(hi/10 == hi%10 && lo/10 == lo%10) printf("%d\n", n);
    }
    return 0;
}

```

此程序中的新知识是 `continue` 和 `break` 语句。`continue` 是指跳回 `for` 循环的开始，执行调整语句并判断循环条件（即“直接进行下一次循环”），而 `break` 是指直接跳出循环^②。

这里的 `continue` 语句的作用是排除不足四位数的 n ，直接检查后面的数。当然，也可以直接从 $x=32$ 开始枚举，但是 `continue` 可以帮助我们偷懒：不必求出循环的起始点。有了 `break`，连循环终点也不必指定——当 n 超过 9999 后会自动退出循环。注意，这里是“退出循环”

^① 这样做，小数部分为 0.5 的数也会受到浮点误差的影响，因此任何一道严密的算法竞赛题目中都需要想办法解决这个问题。后面还会讨论这个问题。

^② 逻辑与“&&”似乎也没有出现过，但假设读者在学习后已经翻阅了相关资料，或者教师已经给学生补充了这个运算符。如果确实没有学过，现在学也来得及。